```
module Document
  # ...
end

class Book
  include Printable
  include Document

  ancestors  # => [Book, Document, Printable, Object, Kernel, BasicObject]
end
```

The implicit receiver of ancestors in the previous code is Book, because in a class definition the role of self is taken by the class. The ancestors chain of Book also contains a third method named print—but Bill is not telling you where it is. If you're curious, you'll have to find it yourself, maybe with some help from your friend irb.

It's almost time to go home after an exhausting but very satisfying day of work. But before you call it a day, Bill does a complete wrap-up of what you learned.

Wrap-Up

Here's a checklist of what you learned today:

- An object is composed of a bunch of instance variables and a link to a class.
- The methods of an object live in the object's class. (From the point of view of the class, they're called *instance methods*.)
- The class itself is just an object of class Class. The name of the class is just a constant.
- Class is a subclass of Module. A module is basically a package of methods. In addition to that, a class can also be instantiated (with new) or arranged in a hierarchy (through its superclass).
- Constants are arranged in a tree similar to a file system, where the names of modules and classes play the part of directories and regular constants play the part of files.
- Each class has an ancestors chain, beginning with the class itself and going up to BasicObject.